

```
DROP FUNCTION IF EXISTS calcular_importe_llamada;
CREATE FUNCTION calcular_importe_llamada(p_id_llamada NUMERIC(11))
RETURNS DECIMAL(11,2)
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE v_duracion DECIMAL(11,2);
    DECLARE v_tipo VARCHAR(4);
    DECLARE v_establecimiento FLOAT;
    DECLARE v_minuto_movil FLOAT;
    DECLARE v_minuto_fijo FLOAT;
    DECLARE v_destino VARCHAR(20);
    DECLARE v_importe DECIMAL(11,2);
    DECLARE v_coste_minuto FLOAT;

    -- Obtener datos de la llamada y tarifa asociada
    SELECT
        l.duracion,
        l.tipo,
        l.destino,
        t.establecimiento,
        t.minuto_movil,
        t.minuto_fijo
    INTO
        v_duracion,
        v_tipo,
        v_destino,
        v_establecimiento,
        v_minuto_movil,
        v_minuto_fijo
    FROM llamadas l
    JOIN clientes c ON l.id_cliente = c.id_cliente
    JOIN tarifas t ON c.tarifa = t.id_tarifa
    WHERE l.id_llamada = p_id_llamada;

    -- Verificar si la llamada existe
    IF v_duracion IS NULL THEN
        RETURN NULL;
    END IF;

    -- Determinar el coste por minuto según si es móvil o fijo
    -- Móvil: números que empiezan por 6 o 7
    -- Fijo: números que empiezan por 8 o 9
    IF v_destino LIKE '6%' THEN
        SET v_coste_minuto = v_minuto_movil;
    ELSE
        SET v_coste_minuto = v_minuto_fijo;
    END IF;

    -- Calcular importe base: duración * coste_minuto + establecimiento
    SET v_importe = (v_duracion * v_coste_minuto) + v_establecimiento;

    -- Aplicar recargo del 2% si es llamada internacional (tipo 'I')
    IF v_tipo = 'I' THEN
        SET v_importe = v_importe * 1.02;
    END IF;
```

```
RETURN v_importe;
END;

-- 2
CREATE PROCEDURE telefonia.actualizar_importes_llamadas()
BEGIN
    -- Variables para el cursor
    DECLARE v_id_llamada NUMERIC(11);
    DECLARE v_id_cliente NUMERIC(11);
    DECLARE v_nombre_cliente VARCHAR(45);
    DECLARE v_apellidos_cliente VARCHAR(45);
    DECLARE v_importe_calculado DECIMAL(11,2);

    -- Variable para controlar el final del cursor
    DECLARE done INT DEFAULT FALSE;

    -- Declarar el cursor para recorrer todas las llamadas
    DECLARE cursor_llamadas CURSOR FOR
        SELECT l.id_llamada, l.id_cliente, c.nombre, c.apellidos
        FROM llamadas l
        JOIN clientes c ON l.id_cliente = c.id_cliente
        ORDER BY l.id_llamada;

    -- Handler para detectar el final del cursor
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    -- Mensaje de inicio
    SELECT 'Iniciando actualización de importes de llamadas...' AS mensaje;

    -- Abrir el cursor
    OPEN cursor_llamadas;

    -- Loop para recorrer todas las llamadas
    read_loop: LOOP
        -- Obtener la siguiente fila
        FETCH cursor_llamadas INTO v_id_llamada, v_id_cliente, v_nombre_cliente,
        v_apellidos_cliente;

        -- Salir del loop si no hay más filas
        IF done THEN
            LEAVE read_loop;
        END IF;

        -- Calcular el importe usando la función
        SET v_importe_calculado = calcular_importe_llamada(v_id_llamada);

        -- Actualizar el importe en la tabla llamadas
        UPDATE llamadas
        SET importe = v_importe_calculado
        WHERE id_llamada = v_id_llamada;

        -- Mostrar mensaje con información del cliente y gasto
        SELECT CONCAT('El cliente ', v_nombre_cliente, ' ', v_apellidos_cliente,
            ' en la llamada ', v_id_llamada,
            ' tiene un gasto de ', FORMAT(v_importe_calculado, 2), ' euros') AS
            mensaje;
```

```

END LOOP;

-- Cerrar el cursor
CLOSE cursor_llamadas;

-- Mensaje de finalización
SELECT 'Actualización de importes completada.' AS mensaje;

END

-- 3
CREATE PROCEDURE telefonia.visualizar_tarifas_clientes()
BEGIN
    -- Variables para el cursor de tarifas
    DECLARE v_id_tarifa NUMERIC(11);
    DECLARE v_nombre_tarifa VARCHAR(45);
    DECLARE done_tarifas INT DEFAULT FALSE;

    -- Variables para el cursor de clientes
    DECLARE v_id_cliente NUMERIC(11);
    DECLARE v_nombre_cliente VARCHAR(45);
    DECLARE v_apellidos_cliente VARCHAR(45);
    DECLARE v_importe_cliente DECIMAL(11,2);
    DECLARE done_clientes INT DEFAULT FALSE;

    -- Variables para contadores y totales
    DECLARE v_total_clientes INT DEFAULT 0;
    DECLARE v_suma_importes DECIMAL(11,2) DEFAULT 0.00;

    -- Cursor principal: recorre todas las tarifas
    DECLARE cursor_tarifas CURSOR FOR
        SELECT id_tarifa, nombre
        FROM tarifas
        ORDER BY id_tarifa;

    -- Cursor secundario: recorre clientes de una tarifa específica
    DECLARE cursor_clientes CURSOR FOR
        SELECT
            c.id_cliente,
            c.nombre,
            c.apellidos,
            COALESCE(SUM(l.importe), 0) as total_importe
        FROM clientes c
        LEFT JOIN llamadas l ON c.id_cliente = l.id_cliente
        WHERE c.tarifa = v_id_tarifa
        GROUP BY c.id_cliente, c.nombre, c.apellidos
        ORDER BY c.nombre, c.apellidos;

    -- Handlers para detectar el final de cada cursor
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done_tarifas = TRUE;

    -- Mensaje de inicio
    SELECT '===== ' AS separador;
    SELECT 'REPORTE DE TARIFAS Y CLIENTES' AS titulo;
    SELECT '===== ' AS separador;

    -- Abrir cursor de tarifas

```

```

OPEN cursor_tarifas;

-- Loop principal: recorrer tarifas
tarifas_loop: LOOP
    FETCH cursor_tarifas INTO v_id_tarifa, v_nombre_tarifa;

    IF done_tarifas THEN
        LEAVE tarifas_loop;
    END IF;

    -- Reiniciar contadores para cada tarifa
    SET v_total_clientes = 0;
    SET v_suma_importes = 0.00;
    SET done_clientes = FALSE;

    -- Mostrar información de la tarifa
    SELECT '' AS separador;
    SELECT CONCAT('TARIFA: ', v_nombre_tarifa, ' (ID: ', v_id_tarifa, ')') AS info_tarifa;
    SELECT '-----' AS separador;

    -- Abrir cursor de clientes para esta tarifa
    OPEN cursor_clientes;

    -- Handler específico para el cursor de clientes
    BEGIN
        DECLARE CONTINUE HANDLER FOR NOT FOUND SET done_clientes = TRUE;

        -- Loop secundario: recorrer clientes de la tarifa actual
        clientes_loop: LOOP
            FETCH cursor_clientes INTO v_id_cliente, v_nombre_cliente, v_apellidos_cliente,
                v_importe_cliente;

            IF done_clientes THEN
                LEAVE clientes_loop;
            END IF;

            -- Mostrar información del cliente
            SELECT CONCAT('  Cliente: ', v_nombre_cliente, ' ', v_apellidos_cliente,
                ' - Gasto total: ', FORMAT(v_importe_cliente, 2), ' euros') AS
                detalle_cliente;

            -- Acumular totales
            SET v_total_clientes = v_total_clientes + 1;
            SET v_suma_importes = v_suma_importes + v_importe_cliente;

        END LOOP clientes_loop;
    END;

    -- Cerrar cursor de clientes
    CLOSE cursor_clientes;

    -- Mostrar totales de la tarifa
    SELECT '-----' AS separador;
    SELECT CONCAT('TOTAL CLIENTES EN TARIFA "', v_nombre_tarifa, '" : ', v_total_clientes)
    AS total_clientes;
    SELECT CONCAT('SUMA TOTAL DE IMPORTES: ', FORMAT(v_suma_importes, 2), ' euros') AS
    suma_importes;

```

```
SELECT '===== ' AS separador;

END LOOP tarifas_loop;

-- Cerrar cursor de tarifas
CLOSE cursor_tarifas;

-- Mensaje de finalización
SELECT 'REPORTE COMPLETADO' AS fin;

END

-- 4
CREATE TRIGGER `validar_clientes_insert`
BEFORE INSERT ON `clientes` FOR EACH ROW BEGIN
    -- Validar NIF: debe tener exactamente 2 caracteres
    IF LENGTH(NEW.nif) != 2 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: El NIF debe tener exactamente 2 caracteres';
    END IF;

    -- Validar edad: debe estar entre 18 y 100 años
    IF NEW.edad < 18 OR NEW.edad > 100 THEN
        SIGNAL SQLSTATE '45001'
        SET MESSAGE_TEXT = 'Error: La edad debe estar comprendida entre 18 y 100 años';
    END IF;

    -- Validar que el cliente no puede ser padrino de sí mismo
    IF NEW.padrino = NEW.id_cliente THEN
        SIGNAL SQLSTATE '45002'
        SET MESSAGE_TEXT = 'Error: Un cliente no puede ser padrino de sí mismo';
    END IF;

END
```